KU LEUVEN

Advanced Computer Architecture

Lab Report 4

Bert Pyck
Hugo Charmant

ELICT: ELECFS
13 maart 2025
Master in Engineering Technology
Academic year 2024-2025

1.1 Task description

The goal of this report is to compare our findings on memory management. In the first part, we will analyze the effects of coalesced memory access on computation time. Next, we will explore the different types of memory, from global and shared to constant memory, and test our understanding by measuring the differences in matrix multiplications. Finally, we will present our findings, provide a comprehensive conclusion, and visualize the results.

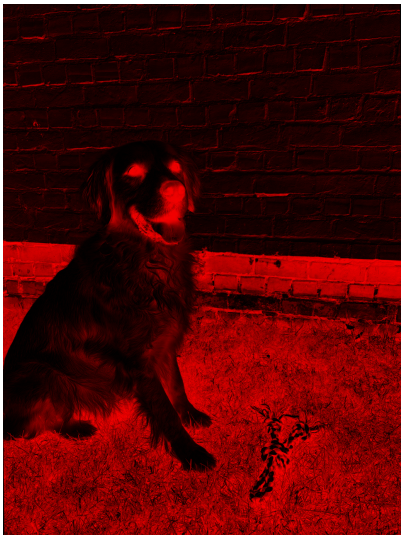
1.2 Introduction

We started the lab session by learning about coalesced memory access, the different types of memory, and the associated challenges. Coalesced memory access means that the threads within a warp (32 threads) can access consecutive memory locations simultaneously, increasing memory throughput. After that, we explored the various kinds of GPU memory in depth: global memory, shared memory, registers, constant memory, texture memory, and local memory.

To demonstrate our understanding, we completed an exercise. The first part consists of measuring the computing time for a kernel that inverts only the red channel of an image. In a first calculation, a non-coalesced memory access is used. Next, a coalesced memory access is performed to show the difference. The expected output can be found in 1.2.



Figuur 1.1: Normal picture



Figuur 1.2: Invert

In the second part analyzed matrix multiplication with different memory models. First, we implemented a kernel that uses only global memory. Then we developed modified versions that utilize shared memory and constant memory. By comparing the execution times and migration times of these different implementations, we were able to gain insight into the influence of memory types on the performance of matrix multiplication.

1.3 Results and conclusion

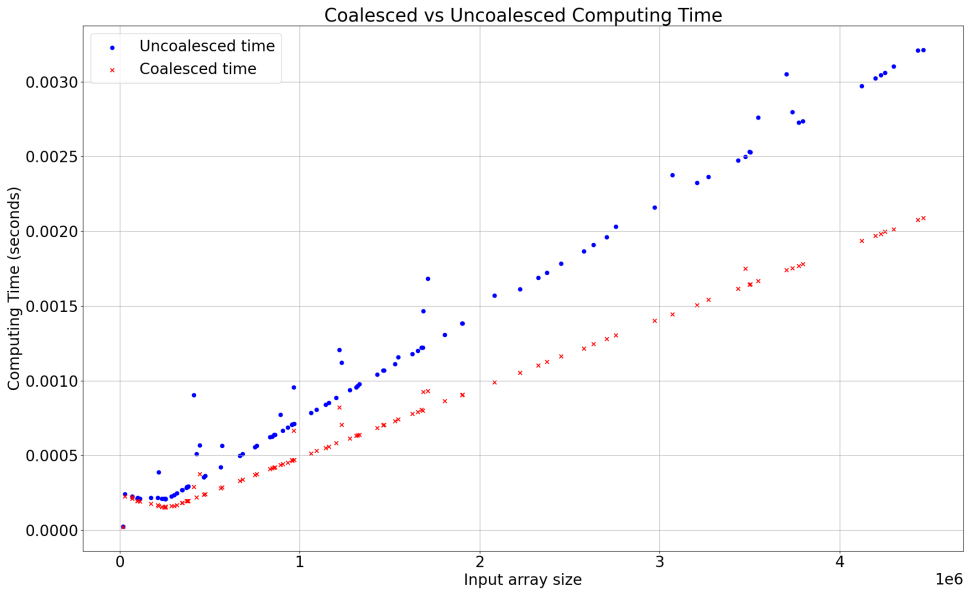
1.3.1 Coalesced compared to uncoalesced memory access

This CUDA-program analyzes the performance difference between coalesced and uncoalesced memory access when modifying the red color channel of an image. It loads an image and represents it as an array of RGB values (bitstream). Two different memory access patterns are tested: uncoalesced memory access, where each thread processes pixels inefficiently by accessing scattered memory locations, and coalesced memory access, where the RGB data is first rearranged into separate contiguous red, green, and blue sections (This is also done in previous labo). This allows threads to access consecutive memory locations efficiently. Both kernels execute on the GPU, and execution times are measured and compared. For random image sizes, the program generates synthetic images and repeats the tests to observe how performance scales with input size. The results can be found in the figure 1.3 and figure 1.4.

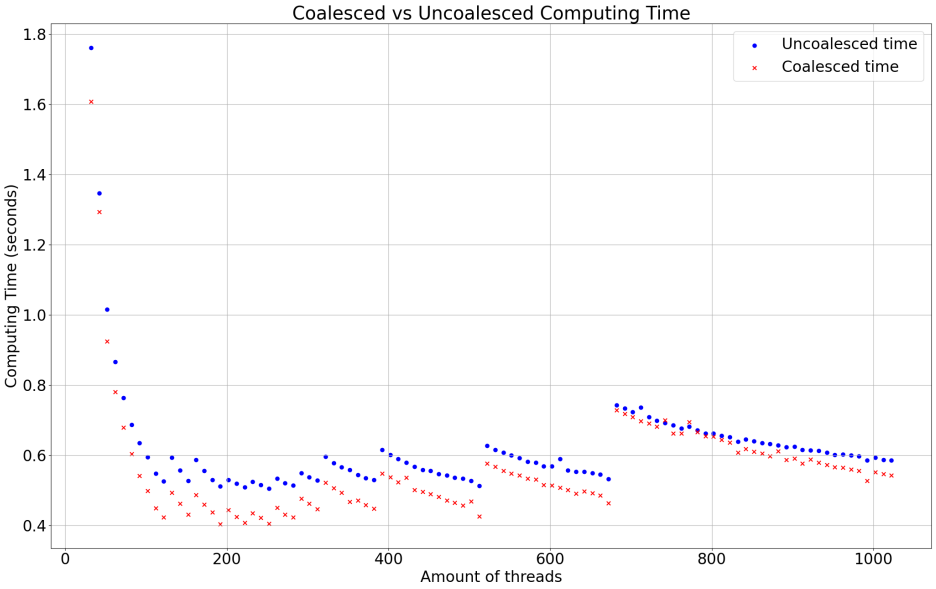
The results match our expectation, the coalesced memory access outperforms the uncoalesced access in computing time.

1.3.2 Matrix multiplication analyzed with different memory models

This CUDA-program compares different memory models for matrix multiplication, analyzing execution time and memory migration costs. It tests three approaches: global memory (matrices are stored in off-chip GPU memory), shared memory (where thread blocks share faster on-chip memory to optimize data access) and constant memory (read-



Figuur 1.3: Effect of input array size on computing time



Figuur 1.4: Effect of block size on computing time

only and optimized for efficient data broadcasting across thread).

The program initializes random matrices and iterates over different matrix sizes then it transfers the matrices from CPU to GPU memory and measures execution time for each multiplication method. Additionally, it tracks the memory migration time which accounts for data transfer from CPU to GPU global memory and from global memory to shared

memory. The kernels execute matrix multiplication by assigning each thread to compute a single element of the output matrix. The global memory kernel directly accesses data, leading to high latency. The shared memory kernel loads data into faster shared memory before computation, reducing redundant memory access. The constant memory kernel utilizes read-only memory to speed up frequent data access.

Experiment 1: Vary Block Sizes (Matrix 16x16)

BlockSize	ConstantKernelTime (ms)	SharedKernelTime (ms)
4	0.0084672	0.0113408
8	0.005168	0.0058048
12	0.0055104	0.0058528
16	0.007104	0.0047488
20	0.0073408	0.0050272
24	0.0078144	0.0059936
28	0.0099424	0.0092096
32	0.0095328	0.0082464

Tabel 1.1: Constant and Shared Kernel Execution Times

BlockSize	AvgCPUtoGPU (ms)	MigrationTime (ms)
4	0.0244496	1.33608e-06
8	0.0116976	6.90251e-07
12	0.0118064	7.00946e-07
16	0.0127888	3.61168e-07
20	0.0122944	3.89963e-07
24	0.0454544	6.09626e-07
28	0.0176752	4.5907e-07
32	0.0287872	4.62361e-07

Tabel 1.2: CPU-GPU Data Transfer and Migration Times

Experiment 2a: Vary Matrix Sizes (Constant Kernel)

MatrixSize	KernelTime (ms)	CPU_to_GPU (ms)
4	0.0042304	0.0123968
8	0.0046368	0.012352
12	0.005216	0.011024
16	0.0064064	0.0114752

Tabel 1.3: Results for Experiment 2a: Vary Matrix Sizes (Constant Kernel)

1.3.3 Experiment 2b: Vary Matrix Sizes (Shared Kernel)

MatrixSize	KernelTime (ms)	CPU_to_GPU (ms)	MigrationTime (ms)
16	0.0052704	0.0121856	3.62814e-07
20	0.0054528	0.0122112	3.89963e-07
24	0.0059296	0.0120000	4.41794e-07
28	0.0067520	0.0120544	5.06787e-07
32	0.0075040	0.0124256	5.41341e-07

Tabel 1.4: Results for Experiment 2b: Vary Matrix Sizes (Shared Kernel)

The results indicate that the optimal block size for both kernels is 16, where the shared memory kernel performs better than the constant memory kernel. Small block sizes (4) result in higher execution times due to overhead, while large block sizes (32) show inefficiencies. As matrix size increases, kernel execution time also increases, but the shared memory kernel maintains better performance. CPU-GPU transfer times remain stable at around 0.012 ms, and migration overhead is negligible. Overall, shared memory improves execution time, especially for larger matrices and well-chosen block sizes.

2 CODE

```
#include <iostream>
#include <cstdint>
#include <cuda_runtime.h>
#include <chrono>

#define OFFSET 16 // Header length
#define AVERAGE 10

int M = 960; // Image width
int N = 1280; // Image height
int C = 3; // Colors (RGB)

double resultFullThreads = 0;
double resultStriding = 0;

int threadsPerBlock = 1024;
int blocksPerGrid;

int totalElements = M * N * C;
double uncoalescedtime = 0;
double coalescedtime = 0;

uint8_t* get_image_array() {
    FILE *imageFile = fopen("./anna.ppm", "rb");
    if (imageFile == NULL) {
        perror("ERROR: Cannot open input file");
        exit(EXIT_FAILURE);
    }

    uint8_t* image_array = (uint8_t*)malloc(M * N * C + OFFSET);
    fread(image_array, sizeof(uint8_t), M * N * C + OFFSET, imageFile);
    fclose(imageFile);

    return image_array + OFFSET;
}

void save_image_array(uint8_t* image_array) {
    FILE *imageFile = fopen("./output_image.ppm", "wb");
    if (imageFile == NULL) {
        perror("ERROR: Cannot open output file");
        exit(EXIT_FAILURE);
    }

    fprintf(imageFile, "P6\n%d %d\n255\n", M, N);
    fwrite(image_array, 1, M * N * C, imageFile);
```

```
    fclose(imageFile);
}


void sortRGBtoRRRGGGBBB(uint8_t* input, uint8_t* output) {
    int totalPixels = M * N; // Total number of pixels (not bytes)

    int redIndex = 0;
    int greenIndex = totalPixels;
    int blueIndex = 2 * totalPixels;

    for (int i = 0; i < totalPixels; i++) {
        output[redIndex++] = input[i * 3]; // Copy Red
        output[greenIndex++] = input[i * 3 + 1]; // Copy Green
        output[blueIndex++] = input[i * 3 + 2]; // Copy Blue
    }
}

void sortRRRGGGBBBtoRGB(uint8_t* input, uint8_t* output) {
    int totalPixels = M * N; // Total number of pixels

    int redIndex = 0;
    int greenIndex = totalPixels;
    int blueIndex = 2 * totalPixels;

    for (int i = 0; i < totalPixels; i++) {
        output[i * 3] = input[redIndex++]; // Copy Red
        output[i * 3 + 1] = input[greenIndex++]; // Copy Green
        output[i * 3 + 2] = input[blueIndex++]; // Copy Blue
    }
}


__global__ void invertRedCoalesced(const uint8_t *input, uint8_t *output, int
    totalElements) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    // totalElements / 3 to only use the red channel RRRR..RGGG..GBBB..
    for (int i = idx; i < totalElements/3; i += blockDim.x * gridDim.x) {
        if (i % 3 == 0) {
            output[i] = 255 - input[i];
        }
    }
}

__global__ void invertRedUncoalesced(const uint8_t *input, uint8_t *output, int
    totalElements) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    // Only invert the red channel
   for (int i = idx; i < totalElements; i += blockDim.x * gridDim.x) {
        if (i % 3 == 0) {
            output[i] = 255 - input[i];
        }
    }
}
```

```
100
101 int main() {
102
103     std::cout<<"Start␣Uncoalesced"<< std::endl;
104
105     for(int seed = 0; seed < 100; seed++){
106
107         if (seed == 0) {
108
109             for(int i = 0; i < AVERAGE; i++){
110                 cudaEvent_t start_cuda, stop_cuda;
111                 cudaEventCreate(&start_cuda);
112                 cudaEventCreate(&stop_cuda);
113
114                 uint8_t* image_array = get_image_array();
115                 uint8_t* invert_array = (uint8_t*)malloc(M * N * C);
116
117                 uint8_t *d_input, *d_output;
118                 cudaMalloc((void**)&d_input, M * N * C * sizeof(uint8_t));
119                 cudaMalloc((void**)&d_output, M * N * C * sizeof(uint8_t));
120
121                 cudaMemcpy(d_input, image_array, M * N * C * sizeof(uint8_t),
                        cudaMemcpyHostToDevice);
122
123                 int gridSize = (totalElements + threadsPerBlock - 1) /
                        threadsPerBlock;
124
125                 cudaEventRecord(start_cuda);
126
127                 invertRedUncoalesced<<<gridSize, threadsPerBlock>>>(d_input,
                        d_output, totalElements);
128                 cudaDeviceSynchronize();
129
130                 cudaEventRecord(stop_cuda);
131                 cudaEventSynchronize(stop_cuda);
132
133                 cudaMemcpy(invert_array, d_output, M * N * C * sizeof(uint8_t),
                        cudaMemcpyDeviceToHost);
134
135                 // Striding have to implement
136
137                 save_image_array(invert_array);
138
139                 cudaFree(d_input);
140                 cudaFree(d_output);
141                 free(image_array - OFFSET);
142                 free(invert_array);
143
144                 float ms;
145                 cudaEventElapsedTime(&ms, start_cuda, stop_cuda);
146                 uncoalescedtime += ms/1000;
147             }
148
149             std::cout<<"Uncoalesced:␣" << (uncoalescedtime/AVERAGE) << "␣sec" <<
                    std::endl;
150
```

```
151
152
153             std::cout<<"Start␣coalesced"<< std::endl;
154
155             for(int i = 0; i < AVERAGE; i++){
156                 cudaEvent_t start_cuda, stop_cuda;
157                 cudaEventCreate(&start_cuda);
158                 cudaEventCreate(&stop_cuda);
159
160                 uint8_t* image_array = get_image_array();
161                 uint8_t* sorted_array = (uint8_t*)malloc(M * N * C); // Create
                        sorted array
162                 uint8_t* processed_array = (uint8_t*)malloc(M * N * C);
163                 uint8_t* invert_array = (uint8_t*)malloc(M * N * C);
164
165                 sortRGBtoRRRGGGBBB(image_array, sorted_array);
166
167                 uint8_t *d_input, *d_output;
168                 cudaMalloc((void**)&d_input, M * N * C * sizeof(uint8_t));
169                 cudaMalloc((void**)&d_output, M * N * C * sizeof(uint8_t));
170
171                 cudaMemcpy(d_input, sorted_array, M * N * C * sizeof(uint8_t),
                        cudaMemcpyHostToDevice);
172
173                 int gridSize = (totalElements + threadsPerBlock - 1) /
                        threadsPerBlock;
174
175                 cudaEventRecord(start_cuda);
176
177                 invertRedCoalesced<<<gridSize, threadsPerBlock>>>(d_input,
                        d_output, totalElements);
178                 cudaDeviceSynchronize();
179
180                 cudaEventRecord(stop_cuda);
181                 cudaEventSynchronize(stop_cuda);
182
183                 cudaMemcpy(processed_array, d_output, M * N * C * sizeof(uint8_t),
                         cudaMemcpyDeviceToHost);
184
185                 sortRRRGGGBBBtoRGB(processed_array, invert_array);
186
187                 save_image_array(invert_array);
188
189                 cudaFree(d_input);
190                 cudaFree(d_output);
191                 free(image_array - OFFSET);
192                 free(sorted_array);
193                 free(processed_array);
194                 free(invert_array);
195
196                 float ms;
197                 cudaEventElapsedTime(&ms, start_cuda, stop_cuda);
198                 coalescedtime += ms/1000;
199             }
200             std::cout<<"Coalesced:␣" << (coalescedtime/AVERAGE) << "␣sec" << std::
                    endl;
```

```
201
202         } else {
203             // For random simulation, choose random dimensions for the image.
204             int minDim = 50;
205             int maxDim = 2500;
206             // Use the current seed to vary the dimensions
207             srand(seed);
208             M = minDim + (rand() % (maxDim - minDim + 1));
209             N = minDim + (rand() % (maxDim - minDim + 1));
210             C = 3;
211             totalElements = M * N * C;
212
213             double localUncoalescedTime = 0;
214             double localCoalescedTime = 0;
215
216             // Uncoalesced timing (AVERAGE iterations)
217             for (int i = 0; i < AVERAGE; i++){
218                 // Allocate and fill a random image array (simulate an image)
219                 uint8_t* image_array = (uint8_t*)malloc(totalElements * sizeof(
                        uint8_t));
220                 for (int j = 0; j < totalElements; j++){
221                     image_array[j] = rand() % 256;
222                 }
223                 uint8_t* invert_array = (uint8_t*)malloc(totalElements * sizeof(
                        uint8_t));
224
225                 uint8_t *d_input, *d_output;
226                 cudaMalloc((void**)&d_input, totalElements * sizeof(uint8_t));
227                 cudaMalloc((void**)&d_output, totalElements * sizeof(uint8_t));
228
229                 cudaMemcpy(d_input, image_array, totalElements * sizeof(uint8_t),
                        cudaMemcpyHostToDevice);
230
231                 int gridSize = (totalElements + threadsPerBlock - 1) /
                        threadsPerBlock;
232                 cudaEvent_t start_cuda, stop_cuda;
233                 cudaEventCreate(&start_cuda);
234                 cudaEventCreate(&stop_cuda);
235                 cudaEventRecord(start_cuda);
236
237                 invertRedUncoalesced<<<gridSize, threadsPerBlock>>>(d_input,
                        d_output, totalElements);
238                 cudaDeviceSynchronize();
239
240                 cudaEventRecord(stop_cuda);
241                 cudaEventSynchronize(stop_cuda);
242
243                 float ms;
244                 cudaEventElapsedTime(&ms, start_cuda, stop_cuda);
245                 localUncoalescedTime += ms / 1000;
246
247                 cudaMemcpy(invert_array, d_output, totalElements * sizeof(uint8_t)
                        , cudaMemcpyDeviceToHost);
248
249                 // Optionally, you can save the image (commented out to avoid many
                         file writes)
```

```
250             // save_image_array(invert_array);
251
252             cudaFree(d_input);
253             cudaFree(d_output);
254             free(image_array);
255             free(invert_array);
256         }
257
258         // Coalesced timing (AVERAGE iterations)
259         for (int i = 0; i < AVERAGE; i++){
260             uint8_t* image_array = (uint8_t*)malloc(totalElements * sizeof(
                    uint8_t));
261             for (int j = 0; j < totalElements; j++){
262                 image_array[j] = rand() % 256;
263             }
264             uint8_t* sorted_array = (uint8_t*)malloc(totalElements * sizeof(
                    uint8_t));
265             uint8_t* processed_array = (uint8_t*)malloc(totalElements * sizeof
                    (uint8_t));
266             uint8_t* invert_array = (uint8_t*)malloc(totalElements * sizeof(
                    uint8_t));
267
268             // Sort from RGBRGB... to RRRGGGBBB format
269             sortRGBtoRRRGGGBBB(image_array, sorted_array);
270
271             uint8_t *d_input, *d_output;
272             cudaMalloc((void**)&d_input, totalElements * sizeof(uint8_t));
273             cudaMalloc((void**)&d_output, totalElements * sizeof(uint8_t));
274             cudaMemcpy(d_input, sorted_array, totalElements * sizeof(uint8_t),
                     cudaMemcpyHostToDevice);
275
276             int gridSize = (totalElements + threadsPerBlock - 1) /
                    threadsPerBlock;
277             cudaEvent_t start_cuda, stop_cuda;
278             cudaEventCreate(&start_cuda);
279             cudaEventCreate(&stop_cuda);
280             cudaEventRecord(start_cuda);
281
282             invertRedCoalesced<<<gridSize, threadsPerBlock>>>(d_input,
                    d_output, totalElements);
283             cudaDeviceSynchronize();
284
285             cudaEventRecord(stop_cuda);
286             cudaEventSynchronize(stop_cuda);
287
288             float ms;
289             cudaEventElapsedTime(&ms, start_cuda, stop_cuda);
290             localCoalescedTime += ms / 1000;
291
292             cudaMemcpy(processed_array, d_output, totalElements * sizeof(
                    uint8_t), cudaMemcpyDeviceToHost);
293
294             // Reorder back from RRRGGGBBB to RGBRGB...
295             sortRRRGGGBBBtoRGB(processed_array, invert_array);
296
297             // Optionally, save the image (commented out)
```

```
            // save_image_array(invert_array);

            cudaFree(d_input);
            cudaFree(d_output);
            free(image_array);
            free(sorted_array);
            free(processed_array);
            free(invert_array);
        }

        std::cout << M*N << "," << (localUncoalescedTime / AVERAGE) << "," <<
            (localCoalescedTime / AVERAGE) << std::endl;

    }
  }
  return 0;
}
```

Listing 2.1: Coalesced vs. Uncoalesced Memory Access

```
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <cuda_runtime.h>
#include <cstdint>

// Define number of iterations for averaging
#define AVERAGE 10

//-------------------------------------------------------
// Constant memory arrays for constant kernel (max 256 elements)
__constant__ uint8_t constA[256];
__constant__ uint8_t constB[256];

//-------------------------------------------------------
// Kernel: Multiply matrices using constant memory
__global__ void multiplyMatrixConstant(uint8_t *C, int n) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    if (row < n && col < n) {
        uint8_t sum = 0;
        for (int k = 0; k < n; k++) {
            sum += constA[row * n + k] * constB[k * n + col];
        }
        C[row * n + col] = sum;
    }
}

//-------------------------------------------------------
// Kernel: Multiply matrices using shared memory and measure global->shared
    migration time
// tile_size is the block dimension (assumes square block)
// Uses dynamic shared memory: size = 2 * tile_size * tile_size * sizeof(uint8_t)
__global__ void multiplyMatrixShared(uint8_t *A, uint8_t *B, uint8_t *C, int n,
    float *d_migrationTime, float clockRate, int tile_size) {
    extern __shared__ uint8_t shared[];
```

```
    uint8_t *tileA = shared;
    uint8_t *tileB = shared + tile_size * tile_size;

    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    uint8_t sum = 0;

    int numTiles = (n + tile_size - 1) / tile_size;
    // For each tile, measure the time to copy from global to shared memory.
    for (int t = 0; t < numTiles; t++) {
        __syncthreads();
        clock_t start = clock();

        int tiledCol = t * tile_size + threadIdx.x;
        int tiledRow = t * tile_size + threadIdx.y;
        if (row < n && tiledCol < n)
            tileA[threadIdx.y * tile_size + threadIdx.x] = A[row * n + tiledCol];
        else
            tileA[threadIdx.y * tile_size + threadIdx.x] = 0;

        if (col < n && tiledRow < n)
            tileB[threadIdx.y * tile_size + threadIdx.x] = B[tiledRow * n + col];
        else
            tileB[threadIdx.y * tile_size + threadIdx.x] = 0;

        __syncthreads();
        clock_t end = clock();
        if (threadIdx.x == 0 && threadIdx.y == 0) {
            d_migrationTime[blockIdx.y * gridDim.x + blockIdx.x] += ((float)(end -
                 start)) / clockRate;
        }

        for (int j = 0; j < tile_size; j++) {
            sum += tileA[threadIdx.y * tile_size + j] * tileB[j * tile_size +
                threadIdx.x];
        }
    }

    if (row < n && col < n)
        C[row * n + col] = sum;
}

//--------------------------------------------------------
// Utility: Initialize a matrix with random 8-bit values
void initializeRandomMatrix(uint8_t *matrix, int size) {
    for (int i = 0; i < size; i++) {
        matrix[i] = rand() % 256;
    }
}

//--------------------------------------------------------
// Main: Run experiments
int main() {
    srand(time(NULL));

    // Set device and get its properties
```

```
89      cudaSetDevice(0);
90      cudaDeviceProp deviceProp;
91      cudaGetDeviceProperties(&deviceProp, 0);
92      // clockRate in Hz (deviceProp.clockRate is in kHz)
93      float clockRate = deviceProp.clockRate * 1000.0f;
94
95      //================================================
96      // Experiment 1: Vary Block Sizes for fixed matrix (16x16)
97      //================================================
98      int fixedN = 16;
99      int blockSizes[] = {4, 8, 12, 16, 20, 24, 28, 32};
100     int numBlockSizes = sizeof(blockSizes) / sizeof(blockSizes[0]);
101
102     std::cout << "Experiment 1: Vary Block Sizes (Matrix " << fixedN << "x" <<
            fixedN << ")\n";
103     std::cout << "BlockSize,ConstantKernelTime(ms),SharedKernelTime(ms),
            AvgCPUtoGPU(ms),MigrationTime(ms)\n";
104
105     for (int i = 0; i < numBlockSizes; i++) {
106         int BS = blockSizes[i];
107         // One-block experiment: gridDim = (1,1) and blockDim = (BS, BS)
108         dim3 blockDim(BS, BS);
109         dim3 gridDim(1, 1);
110         size_t matrixBytes = fixedN * fixedN * sizeof(uint8_t);
111
112         // Allocate and initialize host matrices
113         uint8_t *h_A = (uint8_t*)malloc(matrixBytes);
114         uint8_t *h_B = (uint8_t*)malloc(matrixBytes);
115         uint8_t *h_C = (uint8_t*)malloc(matrixBytes);
116         initializeRandomMatrix(h_A, fixedN * fixedN);
117         initializeRandomMatrix(h_B, fixedN * fixedN);
118
119         float totalConstTime = 0.0f, totalSharedTime = 0.0f;
120         float totalCPU2GPU = 0.0f;
121
122         // Allocate device memory for kernels
123         uint8_t *d_A, *d_B, *d_C;
124         float *d_migrationTime;
125         cudaMalloc((void**)&d_A, matrixBytes);
126         cudaMalloc((void**)&d_B, matrixBytes);
127         cudaMalloc((void**)&d_C, matrixBytes);
128         cudaMalloc((void**)&d_migrationTime, sizeof(float));
129
130         // Run AVERAGE iterations
131         cudaEvent_t start, stop;
132         cudaEventCreate(&start);
133         cudaEventCreate(&stop);
134         for (int iter = 0; iter < AVERAGE; iter++) {
135             //---- Constant Kernel Timing ----
136             cudaEventRecord(start);
137             cudaMemcpyToSymbol(constA, h_A, matrixBytes);
138             cudaMemcpyToSymbol(constB, h_B, matrixBytes);
139             cudaEventRecord(stop);
140             cudaEventSynchronize(stop);
141             float cpu2gpu_const;
142             cudaEventElapsedTime(&cpu2gpu_const, start, stop);
```

```
143
144             cudaEventRecord(start);
145             multiplyMatrixConstant<<<gridDim, blockDim>>>(d_C, fixedN);
146             cudaEventRecord(stop);
147             cudaEventSynchronize(stop);
148             float kernelTimeConst;
149             cudaEventElapsedTime(&kernelTimeConst, start, stop);
150             totalConstTime += kernelTimeConst;
151
152             //---- Shared Kernel Timing ----
153             cudaMemset(d_migrationTime, 0, sizeof(float));
154             cudaEventRecord(start);
155             cudaMemcpy(d_A, h_A, matrixBytes, cudaMemcpyHostToDevice);
156             cudaMemcpy(d_B, h_B, matrixBytes, cudaMemcpyHostToDevice);
157             cudaEventRecord(stop);
158             cudaEventSynchronize(stop);
159             float cpu2gpu_shared;
160             cudaEventElapsedTime(&cpu2gpu_shared, start, stop);
161
162             // Dynamic shared memory: 2 * BS * BS * sizeof(uint8_t)
163             cudaEventRecord(start);
164             multiplyMatrixShared<<<gridDim, blockDim, 2 * BS * BS * sizeof(uint8_t
                    )>>>(d_A, d_B, d_C, fixedN, d_migrationTime, clockRate, BS);
165             cudaEventRecord(stop);
166             cudaEventSynchronize(stop);
167             float kernelTimeShared;
168             cudaEventElapsedTime(&kernelTimeShared, start, stop);
169             totalSharedTime += kernelTimeShared;
170
171             totalCPU2GPU += (cpu2gpu_const + cpu2gpu_shared) / 2.0f;
172         }
173         float avgConstTime = totalConstTime / AVERAGE;
174         float avgSharedTime = totalSharedTime / AVERAGE;
175         float avgCPU2GPU = totalCPU2GPU / AVERAGE;
176
177         // Get migration time from device (only one block)
178         float migrationTime;
179         cudaMemcpy(&migrationTime, d_migrationTime, sizeof(float),
                cudaMemcpyDeviceToHost);
180
181         std::cout << BS << ",␣" << avgConstTime << ",␣" << avgSharedTime << ",␣"
                << avgCPU2GPU << ",␣" << migrationTime << "\n";
182
183         // Cleanup for this iteration
184         cudaFree(d_A);
185         cudaFree(d_B);
186         cudaFree(d_C);
187         cudaFree(d_migrationTime);
188         free(h_A);
189         free(h_B);
190         free(h_C);
191         cudaEventDestroy(start);
192         cudaEventDestroy(stop);
193     }
194
195     //============================================
```

```
    // Experiment 2a: Vary Matrix Sizes for constant kernel (n <= 16)
    //================================================
    int matrixSizesConst[] = {4, 8, 12, 16};
    int numMatrixConst = sizeof(matrixSizesConst) / sizeof(matrixSizesConst[0]);
    std::cout << "\nExperiment 2a: Vary Matrix Sizes (Constant Kernel)\n";
    std::cout << "MatrixSize,KernelTime(ms),CPU_to_GPU(ms)\n";
    for (int i = 0; i < numMatrixConst; i++) {
        int n = matrixSizesConst[i];
        size_t bytes = n * n * sizeof(uint8_t);

        // For one block, set blockDim = (n, n)
        dim3 blockDim(n, n);
        dim3 gridDim(1, 1);

        uint8_t *h_A = (uint8_t*)malloc(bytes);
        uint8_t *h_B = (uint8_t*)malloc(bytes);
        uint8_t *h_C = (uint8_t*)malloc(bytes);
        initializeRandomMatrix(h_A, n * n);
        initializeRandomMatrix(h_B, n * n);

        float totalKernelTime = 0.0f, totalCPU2GPU = 0.0f;

        uint8_t *d_C;
        cudaMalloc((void**)&d_C, bytes);

        cudaEvent_t start, stop;
        cudaEventCreate(&start);
        cudaEventCreate(&stop);
        for (int iter = 0; iter < AVERAGE; iter++) {
            cudaEventRecord(start);
            cudaMemcpyToSymbol(constA, h_A, bytes);
            cudaMemcpyToSymbol(constB, h_B, bytes);
            cudaEventRecord(stop);
            cudaEventSynchronize(stop);
            float t_cpu2gpu;
            cudaEventElapsedTime(&t_cpu2gpu, start, stop);
            totalCPU2GPU += t_cpu2gpu;

            cudaEventRecord(start);
            multiplyMatrixConstant<<<gridDim, blockDim>>>(d_C, n);
            cudaEventRecord(stop);
            cudaEventSynchronize(stop);
            float t_kernel;
            cudaEventElapsedTime(&t_kernel, start, stop);
            totalKernelTime += t_kernel;
        }
        float avgKernelTime = totalKernelTime / AVERAGE;
        float avgCPU2GPU = totalCPU2GPU / AVERAGE;
        std::cout << n << ", " << avgKernelTime << ", " << avgCPU2GPU << "\n";

        cudaFree(d_C);
        free(h_A);
        free(h_B);
        free(h_C);
        cudaEventDestroy(start);
        cudaEventDestroy(stop);
```

```
252     }
253
254     //================================================
255     // Experiment 2b: Vary Matrix Sizes for shared kernel (one block: n <= 32)
256     //================================================
257     int matrixSizesShared[] = {16, 20, 24, 28, 32};
258     int numMatrixShared = sizeof(matrixSizesShared) / sizeof(matrixSizesShared
            [0]);
259     std::cout << "\nExperiment 2b: Vary Matrix Sizes (Shared Kernel)\n";
260     std::cout << "MatrixSize,KernelTime(ms),CPU_to_GPU(ms),MigrationTime(ms)\n";
261     for (int i = 0; i < numMatrixShared; i++) {
262         int n = matrixSizesShared[i];
263         size_t bytes = n * n * sizeof(uint8_t);
264
265         // For one block, blockDim = (n, n)
266         dim3 blockDim(n, n);
267         dim3 gridDim(1, 1);
268
269         uint8_t *h_A = (uint8_t*)malloc(bytes);
270         uint8_t *h_B = (uint8_t*)malloc(bytes);
271         uint8_t *h_C = (uint8_t*)malloc(bytes);
272         initializeRandomMatrix(h_A, n * n);
273         initializeRandomMatrix(h_B, n * n);
274
275         float totalKernelTime = 0.0f, totalCPU2GPU = 0.0f;
276         float totalMigrationTime = 0.0f;
277
278         uint8_t *d_A, *d_B, *d_C;
279         float *d_migrationTime;
280         cudaMalloc((void**)&d_A, bytes);
281         cudaMalloc((void**)&d_B, bytes);
282         cudaMalloc((void**)&d_C, bytes);
283         cudaMalloc((void**)&d_migrationTime, sizeof(float));
284         cudaMemset(d_migrationTime, 0, sizeof(float));
285
286         cudaEvent_t start, stop;
287         cudaEventCreate(&start);
288         cudaEventCreate(&stop);
289         for (int iter = 0; iter < AVERAGE; iter++) {
290             cudaMemset(d_migrationTime, 0, sizeof(float));
291             cudaEventRecord(start);
292             cudaMemcpy(d_A, h_A, bytes, cudaMemcpyHostToDevice);
293             cudaMemcpy(d_B, h_B, bytes, cudaMemcpyHostToDevice);
294             cudaEventRecord(stop);
295             cudaEventSynchronize(stop);
296             float t_cpu2gpu;
297             cudaEventElapsedTime(&t_cpu2gpu, start, stop);
298             totalCPU2GPU += t_cpu2gpu;
299
300             // Dynamic shared memory: 2 * n * n * sizeof(uint8_t)
301             cudaEventRecord(start);
302             multiplyMatrixShared<<<gridDim, blockDim, 2 * n * n * sizeof(uint8_t)
                    >>>(d_A, d_B, d_C, n, d_migrationTime, clockRate, n);
303             cudaEventRecord(stop);
304             cudaEventSynchronize(stop);
305             float t_kernel;
```

```
306             cudaEventElapsedTime(&t_kernel, start, stop);
307             totalKernelTime += t_kernel;
308         }
309         cudaMemcpy(&totalMigrationTime, d_migrationTime, sizeof(float),
                cudaMemcpyDeviceToHost);
310
311         float avgKernelTime = totalKernelTime / AVERAGE;
312         float avgCPU2GPU = totalCPU2GPU / AVERAGE;
313
314         std::cout << n << ",␣" << avgKernelTime << ",␣" << avgCPU2GPU << ",␣" <<
                totalMigrationTime << "\n";
315
316         cudaFree(d_A);
317         cudaFree(d_B);
318         cudaFree(d_C);
319         cudaFree(d_migrationTime);
320         free(h_A);
321         free(h_B);
322         free(h_C);
323         cudaEventDestroy(start);
324         cudaEventDestroy(stop);
325     }
326
327     cudaDeviceReset();
328     return 0;
329 }
```

Listing 2.2: Matrix Multiplication Analyzed with Different Memory Models